Практическая работа №7(1)

Соединение с сервером, работа в Python.

Возможности сервера позволяют соединяться внешним источником для получения информации и визуализации данных в режиме реального времени. Прежде чем приступить к соединению необходимо разрешить его на стороне сервера. В случае с системой beget это необходимо проделать с конкретной базой данных, назначив ей пароль и возможность доступа со сторонних адресов.

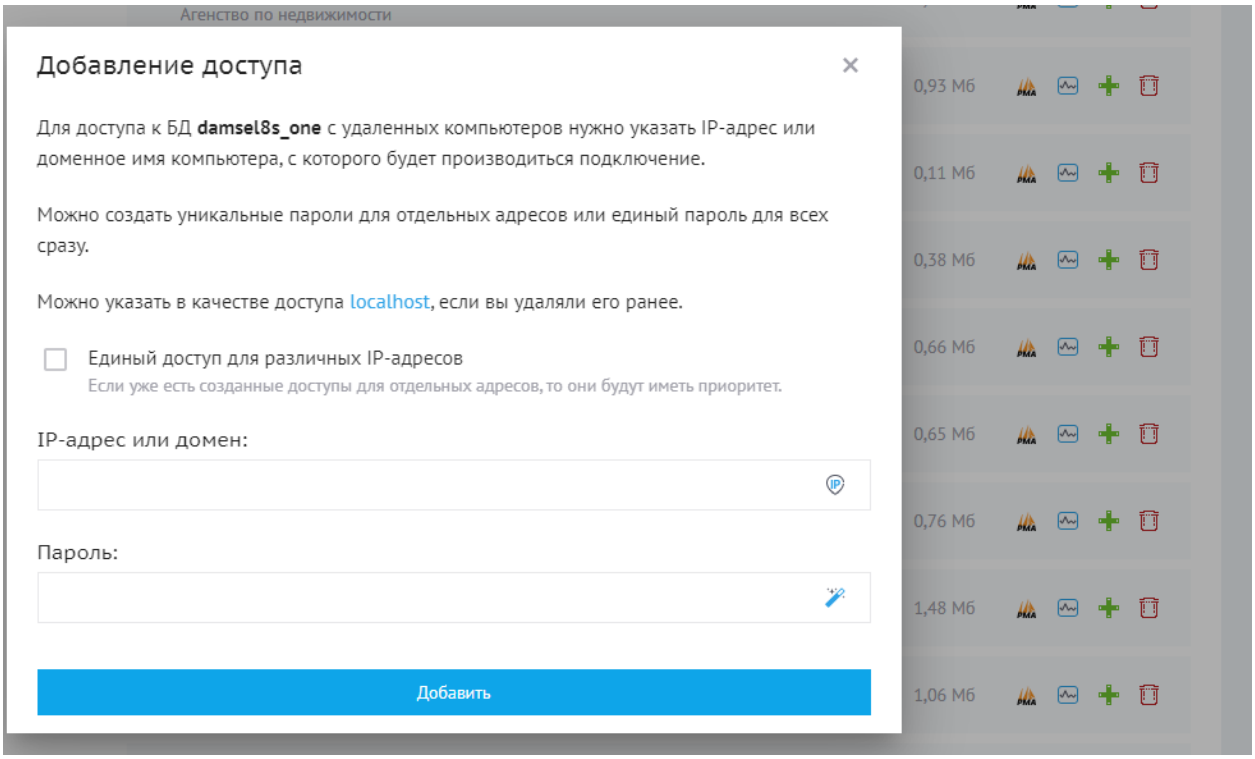


Рисунок 1 – Настройка доступа к базе

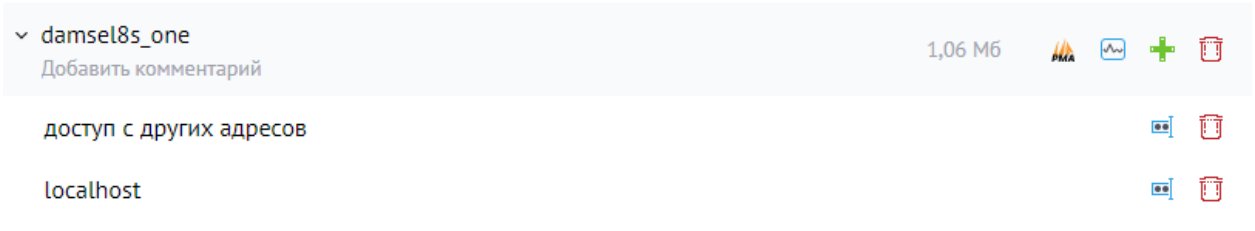


Рисунок 2 – Проверка доступа к базе

Для внешнего подключения можно рассмотреть использование языка и библиотек языка **python**.

В примере будет использован **Google Colab**

Для того, чтобы соединиться с сервером необходимо воспользоваться библиотекой **mysql-connector-python.** Также для дальнейшего использования графиков необходимо загрузить библиотеку **matplotlib.**

Далее необходимо соединиться с сервером, вбив все необходимые параметры. Одним из важнейших для работы является объект cursor. Для успешного выполнения

запроса необходимо воспользоваться методом execute(). В конце работы с запросом необходимо закрыть работу с курсором и соединением.

```
pip install mysql-connector-python
```

```
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.0.0-cp310-cp310-manylinux_2_17_x86_64.whl.metadata (2.0 kB)
Downloading mysql_connector_python-9.0.0-cp310-cp310-manylinux_2_17_x86_64.whl (19.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19.3/19.3 MB 21.6 MB/s eta 0:00:00
Installing collected packages: mysql-connector-python
Successfully installed mysql-connector-python-9.0.0
```

Рисунок 3 – Установка mysql-connector-python

```
import mysql.connector
cnx= mysql.connector.connect(user='совпадает с именем БД', password='пароль', host='адрес хоста', database='название БД')
cursor=cnx.cursor()
query='Select * from air'
cursor.execute(query)

for (id_air, name_air) in cursor:
     print(f"{id_air}, {name_air}")
cursor.close()
cnx.close()
```

```
1, Руслайн air
2, s7 air
3, Уральские авиалинии
4, россия air
5, nord wind air
6, nord avia air
7, Турецкие авиалинии
8, Южный ветер
```

Рисунок 4 – Код обработки соединения с базой и отработки запроса

```
import mysql.connector
import matplotlib.pyplot as plt
cnx=mysql.connector.connect (user='damsel8s_one', password='YvF8QChu', host='damsel8s.beget.tech', database='damsel8s_one')
cursor=cnx.cursor()
query=('SELECT count(avia.date_avia) as count_avia, city.name_city from city inner join avia on city.id_city=avia.id_city group by avia.id_city')
cursor.execute(query)
data=cursor.fetchall()
for row in data:
  print(data)

city = [r[0] for r in data]

count= [r[1] for r in data]


# create a bar chart

plt.bar(city, count)

plt.grid()

plt.xlabel('Города')

plt.ylabel('Количество рейсов')

plt.show()
cursor.close()
cnx.close()
```

Рисунок 5 – Формирование графика на основе данных из запроса

Для успешного формирования графических объектов можно воспользоваться библиотекой matplotlib. Прежде, чем перейти к созданию графических объектов надо занести данные в объект data с помощью метода fetchall() для объекта cursor. Далее идет извлечение необходимых измерений из результата запроса в объекты для будущего построения диаграммы. В качестве диаграммы выбрана столбчатая диаграмма (гистограмма) с указанными измерениями.

```
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
[(4, 'Москва'), (3, 'Калининград'), (3, 'Казань'), (5, 'Адлер'), (3, 'Минеральные воды'), (2, 'Новосибирск'), (1, 'Иркутск'), (6, 'Мурманск'), (2, 'Архангельск')]
```

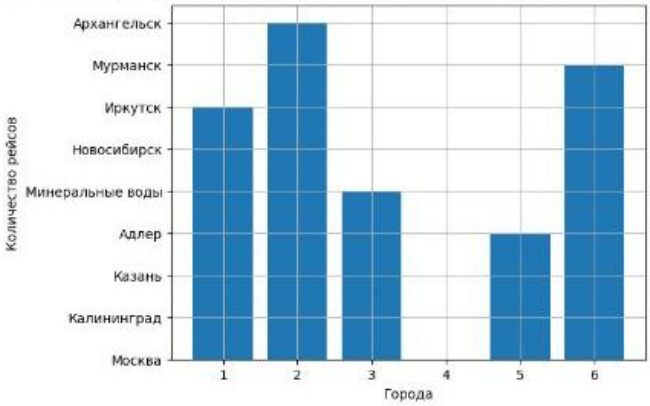


Рисунок 6 – Реализация диаграммы

Также помимо гистограмм можно формировать круговые диаграммы. Помимо выполнения запросов можно вызывать процедуры. На примере ниже показано, как с помощью вызова процедуры и инициализации значений, полученных из процедуры формируется круговая диаграмма.

```
import mysql.connector
import matplotlib.pyplot as plt
cnx=mysql.connector.connect (user='damsel8s_one', password='YvF8QChu', host='damsel8s.beget.tech', database='damsel8s_one')
cursor=cnx.cursor()
cursor.execute("call get_plane_info()")
result=cursor.fetchall()

for row in result:
  print(row[0], row[1])

name_pass = [r[0] for r in result]

price= [r[1] for r in result]

vals=price
labels=name_pass
# create a bar chart

plt.pie(vals, labels=labels)

plt.grid()



plt.show()
cursor.close()
cnx.close()
```

Рисунок 7 – Реализация вызова процедуры и формирования круговой диаграммы

```
Анна Иванова 0.00
Иванов Федор 0.00
Иванов Федор 5348.00
Дмитриев Степан 2518.00
Борисова Полина 7658.00
Сергеев Сергей 2674.00
Волков Игорь 2218.00
Федорова Анна 4530.00
```



Рисунок 8 – Круговая диаграмма и выполнение процедуры

Задание.

Создать соединение с базой данных, а также по 2 гистограммы и круговые диаграммы с помощью библиотек языка python.